Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

8. API Gateway & BFF Pattern . . . 2
8.1. Động lực sử dụng API Gateway . . . 3
8.1.1. Vấn đề: client giao tiếp trực tiếp với nhiều service . . . 3
8.1.2. API Gateway pattern . . . 4
8.1.3. API Gateway và BFF (Backend for Frontend) . . . 4
8.2. Spring Cloud Gateway — Reactive Gateway . . . 6
8.2.1. Vấn đề: chọn công nghệ cho Gateway . . . 6
8.2.2. Kiến trúc Spring Cloud Gateway . . . 6
8.3. Cấu hình route với Eureka . . . 7
8.3.1. Vấn đề: route cố định hay động? . . . 7
8.3.2. Cấu hình route cho LMS Gateway . . . 7
8.3.3. Cách `lb://` hoạt động . . . 8
8.4. Nghiệp vụ cắt ngang (cross-cutting concern) tại Gateway . . . 9
8.4.1. Vấn đề: mỗi service triển khai xác thực, CORS và logging theo cách khác nhau . . . 9
8.4.2. Xác thực — kiểm tra JWT tại Gateway . . . 10
8.4.3. CORS — Cross-Origin Resource Sharing . . . 13
8.4.4. Rate Limiting . . . 14
8.4.5. Logging & Tracing . . . 16
8.5. Case study: KBM . . . 17
8.5.1. Kiến trúc tổng thể . . . 17
8.5.2. Phân tích cấu hình . . . 18
8.5.3. Vấn đề không đồng nhất phiên bản thư viện JWT . . . 18
8.5.4. Lộ trình cải tiến . . . 18
8.6. Tổng kết . . . 20
8.7. Đọc thêm . . . 21
Tài liệu tham khảo . . . 21

CHƯƠNG 8

API Gateway & BFF Pattern

> "A much better approach is for a client to make a single request to an API gateway, a service that serves as the single entry point for API requests into an application from outside the firewall."
> — Chris Richardson, *Microservices Patterns* [2a, Ch.8]

MỤC TIÊU HỌC TẬP

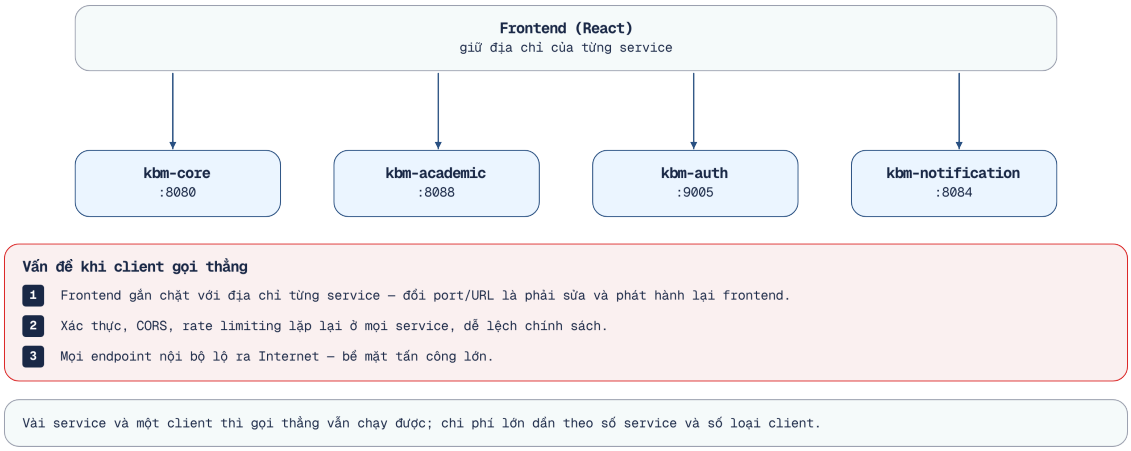
- Hiểu tại sao microservices cần API Gateway và các vấn đề nó giải quyết
- Nắm vững API Gateway pattern và Backend for Frontend (BFF) pattern
- Sử dụng Spring Cloud Gateway (WebFlux) để triển khai gateway
- Cấu hình route với Eureka service discovery (load-balanced URI)
- Thiết kế nghiệp vụ cắt ngang (cross-cutting concern) tại Gateway: xác thực, CORS, rate limiting
- Phân tích kiến trúc gateway trong KBM

8.1. Động lực sử dụng API Gateway

Trong một khuôn viên trường đại học rộng lớn, nếu không có một cổng thông tin duy nhất, việc tìm kiếm và truy cập các dịch vụ sẽ rối rắm. Hệ thống microservices cũng đối mặt với một thách thức tương tự ở phía client.

8.1.1. Vấn đề: client giao tiếp trực tiếp với nhiều service

Khi không có Gateway, client (web, mobile) phải biết địa chỉ của *từng* microservice và gọi trực tiếp. Với KBM gồm nhiều service và module lab, mỗi trang web có thể cần gọi nhiều API khác nhau:



Hình 8.1: *Không có Gateway — client phải biết địa chỉ của từng service*

Việc bắt client (đặc biệt là giao diện người dùng) phải tự ghi nhớ và gọi đến từng service riêng lẻ chẳng khác nào yêu cầu tân sinh viên tự cầm bản đồ đi tìm từng phòng ban để hoàn tất thủ tục nhập học thay vì chỉ đến bộ phận một cửa duy nhất.

Cổng bảo vệ trường đại học

Hãy tưởng tượng hệ thống microservices như một khuôn viên trường đại học với hàng chục tòa nhà (Khoa CNTT, Thư viện, Ký túc xá). Nếu không có cổng chính (API Gateway), một sinh viên sẽ phải tự tìm kiếm và di chuyển đến từng tòa nhà, gõ cửa hỏi đường, và tòa nhà nào cũng phải bố trí bảo vệ riêng để kiểm tra thẻ. **API Gateway** hoạt động như bộ phận một cửa của trường đại học. Sinh viên chỉ cần đến một địa chỉ duy nhất (một URL), nhân viên trực sẽ kiểm tra thẻ sinh viên (authentication) và chuyển yêu cầu đến đúng phòng ban chức năng (routing). Tuy nhiên, phòng ban đích vẫn phải kiểm tra con dấu hoặc giấy giới thiệu theo mức độ rủi ro, rồi tự quyết định sinh viên có quyền thực hiện nghiệp vụ cụ thể hay không (phân quyền, authorization).

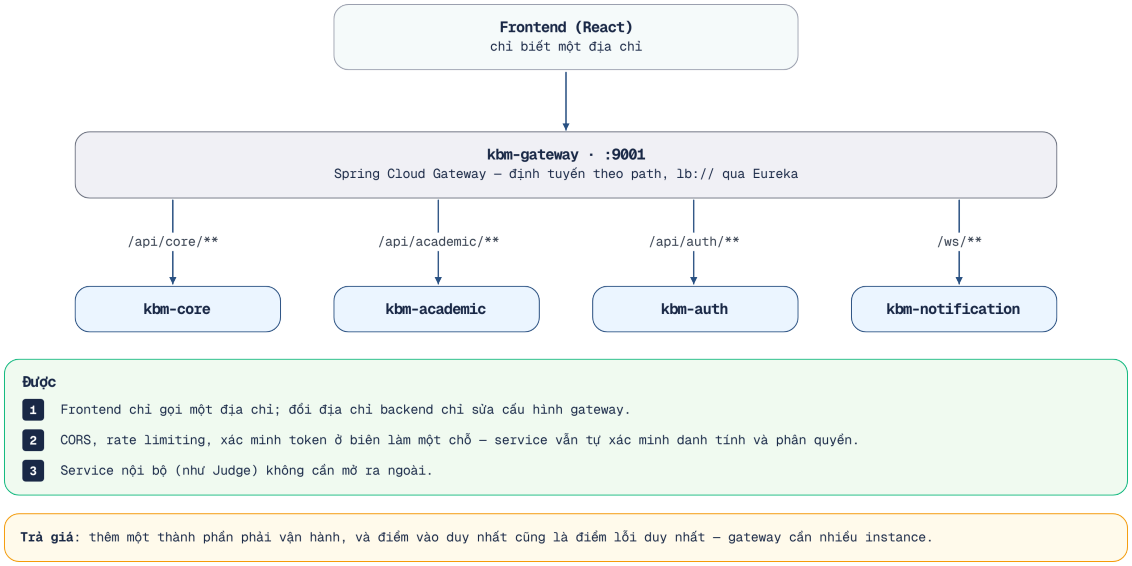
Việc ứng dụng client gọi trực tiếp đến từng microservice gây ra năm vấn đề. Richardson trong [2a, Ch.8] nêu các hạn chế về số lượng request, tính đóng gói và giao thức; phần còn lại là các vấn đề vận hành thường gặp:

1. **Nhiều endpoint:** Client phải quản lý và ghi nhớ URL của mọi dịch vụ riêng lẻ, dẫn đến sự liên kết chặt chẽ (tight coupling) giữa frontend và backend.
2. **Giao thức khác nhau:** Các dịch vụ có thể sử dụng đa dạng giao thức (REST, gRPC, WebSocket), buộc frontend phải trang bị nhiều thư viện hỗ trợ tương ứng, làm tăng độ phức tạp của mã nguồn.

3. **Nghiệp vụ cắt ngang phân tán:** Mỗi dịch vụ phải tự triển khai các nghiệp vụ cắt ngang như xác thực, CORS và rate limiting, gây ra sự trùng lặp và nguy cơ thiếu nhất quán trong các chính sách hệ thống.
4. **Mạng không an toàn:** Việc phơi bày trực tiếp toàn bộ API endpoint nội bộ ra môi trường Internet làm gia tăng bề mặt tấn công (attack surface).
5. **API không phù hợp:** Các giao diện lập trình nội bộ thường được thiết kế cho quá trình tương tác giữa máy chủ với máy chủ, trả về lượng dữ liệu dư thừa hoặc yêu cầu quá nhiều vòng gọi mạng, trở thành điểm nghẽn đối với các nền tảng di động có băng thông hạn hẹp.

8.1.2. API Gateway pattern

API Gateway là **điểm vào duy nhất** (single entry point) — toàn bộ yêu cầu (request) từ client sẽ đi qua Gateway và được định tuyến (route) đến đúng dịch vụ tương ứng:



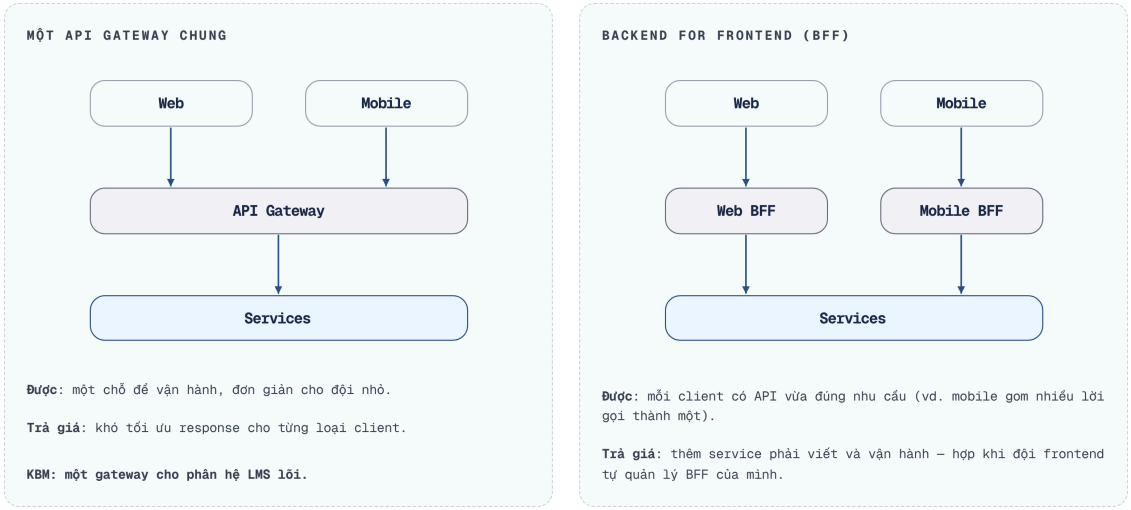
Hình 8.2: API Gateway — điểm vào duy nhất, định tuyến đến từng service

Gateway xử lý tập trung các nghiệp vụ cắt ngang (**cross-cutting concern**), bao gồm: xác thực ở biên (edge authentication), CORS, giới hạn lưu lượng (rate limiting), logging và kết thúc SSL (SSL termination). Các dịch vụ phía sau không cần lặp lại CORS hay rate limiting, nhưng vẫn phải xác minh bằng chứng danh tính theo tầng tin cậy (trust tier) và thực hiện phân quyền cục bộ (local authorization) cho tài nguyên mình sở hữu.

Gateway thường được ví như một "smart edge" (điểm biên thông minh) chuyên xử lý nghiệp vụ cắt ngang. Tuy nhiên, theo đúng tinh thần của nguyên lý **"Smart endpoints and dumb pipes"** do James Lewis và Martin Fowler nêu ra (2014), Gateway không nên chứa business logic. Đối với dữ liệu nghiệp vụ, Gateway vẫn phải duy trì vai trò của một "dumb pipe" (chỉ định tuyến cơ bản) để tránh lặp lại vấn đề của ESB.

8.1.3. API Gateway và BFF (Backend for Frontend)

Richardson trong [2a, Ch.8] phân biệt hai biến thể:



Hình 8.3: API Gateway (một Gateway chung) và BFF (Gateway riêng cho từng client)

Biến thể phổ biến nhất là **API Gateway**, nơi một Gateway duy nhất phục vụ mọi loại client. Phương pháp này đặc biệt phù hợp với các đội ngũ nhỏ hoặc khi các ứng dụng client (như web admin và web sinh viên) có nhu cầu truy xuất dữ liệu tương đồng nhau. Tuy nhiên, khi hệ thống mở rộng, biến thể **BFF (Backend for Frontend)** phù hợp hơn: mỗi loại client có một Gateway riêng. BFF thường cần thiết khi ứng dụng di động (mobile app) đòi hỏi định dạng API khác biệt so với nền tảng web, ví dụ như cần gộp nhiều lời gọi (batching) và tối giản hóa dữ liệu trả về để tiết kiệm băng thông.

KBM sử dụng **một API Gateway duy nhất** cho LMS core — điều này hoàn toàn phù hợp vì các web frontend chính đều có chung những yêu cầu API tương tự nhau. Với DevOps Lab, KBM bổ sung một lớp router riêng theo hostname để điều hướng workspace/lab runtime; đây là gateway chuyên biệt cho một Bounded Context, không thay thế Spring Cloud Gateway của LMS core.

Khi nào cần BFF? BFF trở nên cần thiết khi các client khác nhau có nhu cầu API hoàn toàn khác biệt về bản chất — không chỉ là "lọc bớt các trường dữ liệu":

Nếu hệ thống chỉ phục vụ web và web admin với nhu cầu dữ liệu tương tự nhau, một API Gateway duy nhất là đủ — áp dụng BFF lúc này chỉ gây dư thừa kỹ thuật (over-engineering).

Khi có thêm ứng dụng di động, một API Gateway duy nhất buộc client mobile gọi nhiều API rời rạc chỉ để dựng một màn hình; một Mobile BFF chuyên gom dữ liệu (aggregate) sẽ cắt đáng kể số vòng gọi mạng.

Ở kịch bản phức tạp nhất — thêm thiết bị IoT (như hệ thống điểm danh khuôn mặt ở giảng đường) và bên thứ ba (như hệ thống quản lý đào tạo của trường cần đồng bộ điểm) — một Gateway duy nhất sẽ phình to, khó bảo trì; đây là lúc nên chia thành **nhiều BFF độc lập** phục vụ riêng từng nhóm client.

Ví dụ: nếu KBM phát triển thêm **mobile app** cho sinh viên, nền tảng này sẽ yêu cầu: (1) **batched API** — giao diện "Bảng điều khiển" (Dashboard) chỉ cần một lời gọi để lấy cả thông tin cá nhân, bài nộp gần đây và thứ hạng (thay vì phải gọi ba lần, nhờ đó giảm ảnh hưởng của mạng di động chậm); (2) **reduced payload** — thiết bị di động không cần dữ liệu định dạng sẵn cho HTML mà chỉ

cần dữ liệu thô gọn nhẹ; (3) **tích hợp thông báo đẩy (push notification)** — Gateway dành riêng cho di động sẽ quản lý các mã định danh thiết bị (device token).

Newman trong [4a, Ch.4] khuyến nghị: **đội ngũ phát triển giao diện (frontend)** nên quản lý BFF — đội ngũ phát triển ứng dụng di động chịu trách nhiệm về Mobile BFF, đội ngũ phát triển web đảm nhận Web BFF. Mỗi BFF hoạt động như một lớp xử lý mỏng (thin layer): tiếp nhận yêu cầu từ client, gọi các dịch vụ nội bộ (downstream service), tiến hành gom nhóm và biến đổi dữ liệu (aggregate và transform), sau đó trả về định dạng phù hợp cho từng loại client.

⚠ Aggregation vs Orchestration

BFF được phép thực hiện *Data Aggregation* (gom nhiều lời gọi API lại để giao diện người dùng (UI) hiển thị nhanh hơn). Tuy nhiên, cần đặc biệt tránh việc sử dụng BFF hay Gateway để thực hiện *Business Orchestration* (điều phối nghiệp vụ, ví dụ: tạo bài nộp rồi cập nhật điểm số). Mọi logic điều phối giao dịch phải nằm ở tầng dịch vụ miền (sử dụng Saga/Process Manager như ở Chương 6).

8.2. Spring Cloud Gateway — Reactive Gateway

Khi xây dựng API Gateway, việc lựa chọn công nghệ lõi ảnh hưởng trực tiếp đến khả năng mở rộng của hệ thống.

8.2.1. Vấn đề: chọn công nghệ cho Gateway

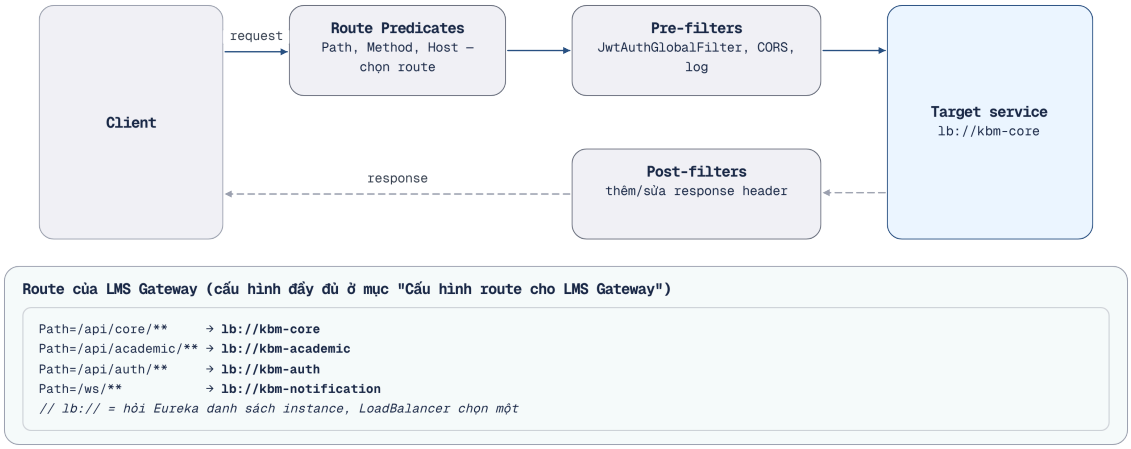
Trong hệ sinh thái Spring, lập trình viên thường cân nhắc giữa hai lựa chọn chính khi xây dựng Gateway tập trung:

- **Mô hình hoạt động**: Spring Cloud Gateway sử dụng mô hình phản ứng (Reactive, WebFlux, non-blocking), cho phép xử lý số lượng lớn các kết nối đồng thời với rất ít luồng (thread). Trong khi đó, Netflix Zuul (phiên bản 1.x) dựa trên mô hình Servlet truyền thống (blocking, thread-per-request), dễ trở thành điểm nghẽn khi tải lớn.
- **Hiệu năng và WebSocket**: Nhờ kiến trúc WebFlux, Spring Cloud Gateway giữ được nhiều kết nối và chờ I/O với ít thread hơn, đồng thời hỗ trợ sẵn WebSocket (native support) — một tính năng cốt lõi cho các ứng dụng thời gian thực. Ngược lại, Zuul 1.x có hiệu năng thấp hơn do phụ thuộc vào giới hạn của thread pool và hoàn toàn không hỗ trợ WebSocket.
- **Hệ sinh thái và tình trạng**: Spring Cloud Gateway hiện là dự án đang được tích cực phát triển (active), được Spring chính thức khuyên dùng và gắn kết chặt chẽ vào hệ sinh thái Spring Cloud. Trái lại, module Zuul 1.x của Spring Cloud Netflix đã chuyển sang chế độ bảo trì từ năm 2018 và bị gỡ khỏi Spring Cloud từ bản 2020.0, nên hiện chỉ được xem như một công nghệ cũ (legacy).

Dựa trên các ưu điểm này, LMS core sử dụng **Spring Cloud Gateway**. Quyết định này nhằm đáp ứng yêu cầu hỗ trợ giao thức WebSocket cho tính năng đẩy thông báo (notification push) liên tục, đồng thời đảm bảo hiệu năng cao cho hệ thống.

8.2.2. Kiến trúc Spring Cloud Gateway

Khác với các gateway nguyên khối thế hệ trước, Spring Cloud Gateway chia quá trình xử lý request thành những thành phần độc lập và dễ kết hợp. Hình 8.4 minh họa luồng đi của một request: nó lần lượt được đối chiếu với các **Predicate** (điều kiện định tuyến) và đi qua chuỗi **Filter** (bộ lọc) trước khi được chuyển tới service đích.



Hình 8.4: *Kiến trúc Spring Cloud Gateway — Route, Predicate, Filter*

Kiến trúc của Spring Cloud Gateway xoay quanh ba khái niệm cốt lõi:

- **Route**: Quy tắc định tuyến từ một điều kiện đầu vào đến URI đích — ví dụ chuyển toàn bộ `/api/core/**` sang `lb://kbm-core`.
- **Predicate**: Phép thử boolean quyết định request có khớp Route hay không, dựa trên các thuộc tính như `Path` hay `Method`.
- **Filter**: Điểm can thiệp vào vòng đời request/response, chạy trước hoặc sau bước định tuyến để xử lý các nghiệp vụ cắt ngang — xác minh JWT qua `JwtAuthGlobalFilter`, gắn thêm header, ghi log.

Về dependency: Gateway sử dụng thư viện `spring-cloud-starter-gateway` (WebFlux; từ Spring Cloud 2025.0 đổi tên thành `spring-cloud-starter-gateway-server-webflux`, và tiền tố cấu hình `spring.cloud.gateway.` thành `spring.cloud.gateway.server.webflux.`) và `spring-cloud-starter-netflix-eureka-client`. **Lưu ý**: Gateway hoạt động dựa trên WebFlux (mô hình phản ứng, không chặn luồng), nên *không* tương thích với `spring-boot-starter-web` (mô hình servlet, chặn luồng). Khai báo đồng thời cả hai thư viện này trong dự án Gateway sẽ gây xung đột (conflict) và khiến ứng dụng không thể khởi động.

8.3. Cấu hình route với Eureka

8.3.1. Vấn đề: route cố định hay động?

Phương pháp đơn giản nhất là gắn cố định (hardcode) URL cho từng dịch vụ trong tệp cấu hình của Gateway. Tuy nhiên, khi các dịch vụ mở rộng quy mô hoặc được triển khai sang container mới, các URL sẽ thay đổi. Khi đó, việc cập nhật thủ công cấu hình Gateway sẽ khó khăn và dễ phát sinh lỗi.

Giải pháp: kết hợp Gateway routing với **Eureka service discovery** (đã học ở Chương 4). Gateway không cần lưu trữ địa chỉ IP và cổng cụ thể của dịch vụ — nó chỉ cần tên dịch vụ: Eureka cung cấp danh sách instance đang hoạt động, Spring Cloud LoadBalancer chọn một instance trong danh sách đó, và Gateway chuyển tiếp request tới đích đã chọn.

8.3.2. Cấu hình route cho LMS Gateway

Listing 8.1 là cấu hình route của LMS Gateway: mỗi route trỏ tới tên service trong Eureka thay vì địa chỉ cố định.

Listing 8.1: *Cấu hình route của LMS Gateway (YAML + Eureka)*

```yaml
# application-lb.yml — LMS Gateway routing configuration
spring:
  cloud:
    gateway:
      routes:
        # Core Service — questions, submissions, contests
        - id: kbm-core
          uri: lb://kbm-core          # lb:// = Eureka lookup + load balance
          predicates:
            - Path=/api/core/**
          filters:
            - StripPrefix=2               # /api/core/questions → /questions

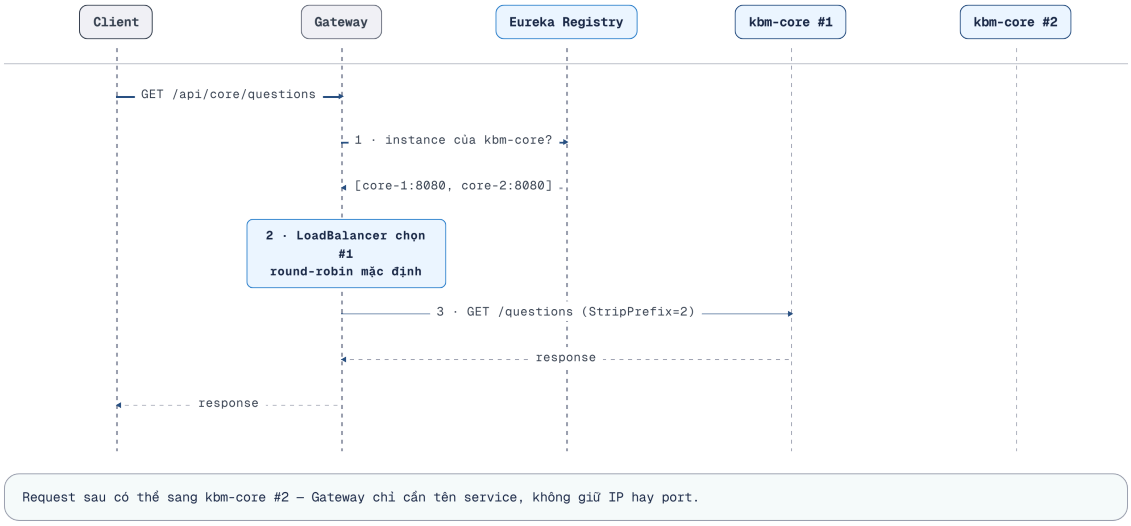
        # Academic Service (kbm-academic) — courses, grades
        - id: kbm-academic
          uri: lb://kbm-academic
          predicates:
            - Path=/api/academic/**
          filters:
            - StripPrefix=2

        # Auth Service — login, register
        - id: kbm-auth
          uri: lb://kbm-auth
          predicates:
            - Path=/api/auth/**
          filters:
            - StripPrefix=2

        # Notification — WebSocket endpoint
        - id: kbm-notification-ws
          uri: lb://kbm-notification
          predicates:
            - Path=/ws/**
```

8.3.3. Cách `lb://` hoạt động

Hình 8.5 minh họa luồng phân giải URI có tiền tố `lb://`: request được định tuyến đến một tên service trong Eureka thay vì một IP tĩnh:



Hình 8.5: *Luồng `lb://` — Eureka lookup + load balance + route*

`lb://kbm-core` thực hiện ba bước tự động:

1. **Lookup**: truy vấn Eureka để lấy danh sách toàn bộ instance đang đăng ký dưới tên `kbm-core`
2. **Load balance**: lựa chọn một instance thông qua Spring Cloud LoadBalancer (round-robin mặc định)
3. **Forward**: chuyển tiếp request đến instance đã được chọn

`StripPrefix=2` loại bỏ hai phần đầu tiên của đường dẫn: `/api/core/questions` → `/questions`. Nhờ đó, dịch vụ không cần quan tâm đến đường dẫn gốc (mount path) của nó tại Gateway — dịch vụ chỉ cần xử lý đường dẫn nội bộ của chính nó.

Service không biết Gateway

Các dịch vụ phía sau *không nên biết* về sự tồn tại của Gateway. Mỗi dịch vụ tự xử lý các đường dẫn nội bộ tương ứng (`/questions`, `/users`), trong khi Gateway đảm nhiệm việc xử lý tiền tố (prefix) và định tuyến. Điều này đảm bảo: (1) các dịch vụ có thể được kiểm thử độc lập (standalone) mà không phụ thuộc vào Gateway; (2) việc cấu trúc lại định tuyến tại Gateway không ảnh hưởng đến mã nguồn của dịch vụ; (3) dịch vụ dễ chuyển sang môi trường khác (portable) — có thể triển khai sau các gateway khác (như NGINX, Kong) mà không cần chỉnh sửa mã nguồn.

8.4. Nghiệp vụ cắt ngang (cross-cutting concern) tại Gateway

Triển khai các nghiệp vụ cắt ngang ở mỗi dịch vụ giống như từng lớp học tự đặt ra quy định kiểm tra thẻ sinh viên riêng lẻ thay vì giao cho bảo vệ cổng trường: vừa dư thừa, vừa không thống nhất và khó quản lý.

8.4.1. Vấn đề: mỗi service triển khai xác thực, CORS và logging theo cách khác nhau

Nếu mỗi dịch vụ tự viết một bộ phân tích JWT, tự cấu hình CORS và tự triển khai rate limiting theo cách riêng, mã nguồn sẽ bị trùng lặp và chính sách dễ trôi lệch. Gateway giải quyết phần chính sách ở biên: tiếp nhận thông tin xác thực (credential) từ client, loại bỏ dữ liệu định danh có thể bị giả mạo, kiểm tra token và áp dụng các biện pháp bảo vệ lưu lượng. Điều đó không miễn trách

nhiệm cho resource service: service đích vẫn xác minh bằng chứng đã ký theo hợp đồng chung và tự thực hiện phân quyền nghiệp vụ.

8.4.2. Xác thực — kiểm tra JWT tại Gateway

Hạn chế của Session Cookie

Trong monolith, trường đại học chỉ có một cô thủ thư duy nhất. Khi một sinh viên vào thư viện, cô nhận ra mặt và nhớ đó là sinh viên (Session Cookie). Nhưng khi trường mở rộng ra 10 chi nhánh thư viện (microservices), sinh viên sang chi nhánh khác, cô thủ thư mới sẽ không biết người đó là ai. Các chi nhánh phải dùng chung một sổ đăng ký trung tâm (Shared Redis), và mọi lượt kiểm tra đều phụ thuộc vào sổ đó. Đó là lý do chúng ta chuyển sang **JWT (JSON Web Token)** — hãy coi nó như **thẻ sinh viên điện tử**. Trên thẻ JWT đã dán ảnh, ghi mã sinh viên và có "con dấu đỏ" chữ ký điện tử của Phòng Đào tạo. Các chi nhánh (downstream service) hoặc cổng bảo vệ (Gateway) chỉ cần nhìn con dấu là biết thẻ thật, hoàn toàn không cần gọi điện hỏi lại Phòng Đào tạo (xác minh không trạng thái, stateless verification).

Do hạn chế của Session Cookie trong kiến trúc phân tán, KBM lựa chọn cơ chế xác thực không trạng thái (stateless) với JSON Web Token (JWT). Gateway là chốt kiểm tra đầu tiên, nhưng kết quả kiểm tra tại đó không phải bằng chứng duy nhất cho mọi trust boundary. Ở đây chỉ cần theo dõi trách nhiệm kiểm tra ở Gateway và service đích; cấu trúc JWT, issuer/audience và các trust tier được giải thích ở §9.2–9.3. Đoạn mã dưới đây mô tả **thiết kế chuyển tiếp** của `JwtAuthGlobalFilter`: loại bỏ các header định danh do client tự khai, xác minh access token tại biên và giữ lại `Authorization` để các service đang xác minh token không bị mất bằng chứng trước khi cơ chế token nội bộ có chữ ký được triển khai:

Listing 8.2: `GlobalFilter` xác minh JWT tại Gateway

```
// Gateway JWT Filter — validate token trước khi route
@Component
public class JwtAuthGlobalFilter implements GlobalFilter, Ordered {

    private final JwtUtil jwtUtil;

    public JwtAuthGlobalFilter(JwtUtil jwtUtil) {
        this.jwtUtil = jwtUtil;
    }

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        ServerHttpRequest request = exchange.getRequest();
        String path = request.getURI().getPath();

        // Bắt buộc XÓA identity headers do client tự gửi (chống Header Spoofing).
        // KHÔNG xóa Authorization trước khi có signed downstream token thay thế.
        ServerHttpRequest.Builder requestBuilder = request.mutate()
            .headers(headers -> {
                // Gỡ mọi X-User-* do client gửi, không phân biệt hoa thường
                headers.keySet().removeIf(
                        name -> name.regionMatches(true, 0, "X-User-", 0, 7));
            });

        // Bỏ qua xác thực cho các endpoint công khai
```

```
        if (isPublicEndpoint(path)) {
            return chain.filter(
                exchange.mutate().request(requestBuilder.build()).build()
            );
        }

        // Xác minh alg + signature + iss + aud + exp + nbf cho audience của Gateway
        String token = extractToken(request);
        if (token == null || !jwtUtil.validateAccessToken(token, "kbm-gateway")) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        // Context dẫn xuất: chỉ dùng cho routing/rate limit.
        // Giữ Authorization cho downstream verifier.
        requestBuilder.headers(headers -> {
            headers.set("X-User-Id", jwtUtil.getUserId(token));
            headers.set("X-User-Roles", jwtUtil.getRoles(token));
        });

        return chain.filter(
            exchange.mutate().request(requestBuilder.build()).build()
        );
    }

    @Override
    public int getOrder() {
        return -1; // Chạy TRƯỚC RequestRateLimiter
    }
}
```

Sau khi Gateway xác minh JWT, thông tin định danh (claim) cần được truyền đến các dịch vụ phía sau (downstream) mà không biến một header thuần thành bằng chứng danh tính. Hình 8.6 mô tả luồng chuyển tiếp: token đã ký được giữ lại hoặc được đổi thành token nội bộ đúng audience; các header dẫn xuất chỉ là context tiện dụng cho routing, logging và rate limiting:



Hình 8.6: Luồng JWT tại Gateway — bằng chứng có chữ ký và context định danh dẫn xuất

Tại thời điểm khảo sát (đầu năm 2026), Gateway của KBM chèn `X-User-Id` và `X-User-Roles`; một số service tin các header này, trong khi một số service khác tự parse JWT bằng thư viện và phiên bản khác nhau. Đây là sự trôi lệch triển khai (implementation drift), không phải phòng thủ chiều sâu (defense in depth) có chủ đích. Việc tin header thuần chỉ được ghi nhận là **rủi ro được chấp nhận** (accepted risk) tạm thời khi service không thể bị truy cập ngoài đường đi đã kiểm soát; đó không phải kiến trúc mục tiêu.

Ở **mục tiêu Tier 1** (§9.3), Gateway xác minh token ngoài rồi truyền bằng chứng có chữ ký xuống service đích: giữ access token trong giai đoạn chuyển tiếp, hoặc dùng token exchange để đổi sang token nội bộ đúng audience. Resource service xác minh issuer, audience, signature và freshness trước khi dùng claim, sau đó tự kiểm tra phân quyền nghiệp vụ. `X-User-*` có thể tồn tại như dữ liệu dẫn xuất, nhưng không được dùng một mình để ra quyết định cấp quyền.

⚠ Giả định tin cậy mạng

Chính sách mạng (quy tắc tường lửa, Kubernetes NetworkPolicy) phải chặn truy cập trực tiếp vào service, còn Gateway phải loại bỏ mọi `X-User-*` từ Internet trước khi tự ghi đè context đã dẫn xuất. Đây là điều kiện tối thiểu để chấp nhận rủi ro nói trên, nhưng chỉ chứng minh *đường đi của request*, không chứng minh nội dung claim.

Cần nhớ giới hạn của mTLS: nó bảo vệ kênh truyền và xác thực *workload/actor* ở hai đầu, không xác thực *user subject* trong `X-User-Id`. Một workload nội bộ bị chiếm quyền vẫn có thể tạo header giả. Vì vậy, request nhạy cảm cần đồng thời có bằng chứng danh tính có chữ ký (signed identity proof) đúng audience, phân quyền cục bộ và — nếu service gọi thay mặt người dùng — ngữ nghĩa delegation tách biệt subject khỏi actor.

8.4.3. CORS — Cross-Origin Resource Sharing

Gateway là **nơi duy nhất** quản lý cấu hình CORS: các origin, method và header được chấp nhận. Cấu hình `globalcors` trong Spring Cloud Gateway cho phép khai báo `allowedOrigins` (các origin hợp lệ), `allowedMethods`, `allowedHeaders`, `allowCredentials` — các dịch vụ phía sau không cần thiết lập cấu hình CORS vì Gateway đã đảm nhận chức năng này.

__Listing 8.3:__ Cấu hình CORS an toàn tại Gateway

```yaml
spring:
  cloud:
    gateway:
      globalcors:
        cors-configurations:
          '[/**]':
            allowed-origins:
              - "https://kbm.example.edu"
              - "https://admin.kbm.example.edu"
            allowed-methods: "GET, POST, PUT, DELETE, OPTIONS"
            # BẮT BUỘC — thiếu dòng này thì allowedHeaders = null
            # → checkHeaders() trả null → preflight bị từ chối
            # → mọi lời gọi API cross-origin kèm JWT bị chặn ngay
            allowed-headers:
              - "Authorization"
              - "Content-Type"
              - "X-Request-ID"
            # chỉ đổi thành true khi xác thực bằng cookie
            # và đã có biện pháp chống CSRF
            allow-credentials: false
```

Cấu hình CORS ở cấp độ Gateway giúp hệ thống tránh phải thiết lập lại các quy tắc tương tự ở từng dịch vụ. Bằng cách khai báo tập trung các origin hợp lệ (`allowedOrigins`), method được phép (`allowedMethods`), danh sách header được chấp nhận (`allowedHeaders`) và việc có cho phép gửi kèm credential hay không (`allowCredentials`), Gateway kiểm soát nhất quán các yêu cầu cross-origin. **Lưu ý quan trọng**: Spring Cloud Gateway *không* tự động cho phép mọi header khi khai báo `globalcors` qua YAML — nếu thiếu `allowed-headers`, `CorsConfiguration.checkHeaders()` sẽ trả về `null` (từ chối), khiến mọi preflight request chứa `Authorization` thất bại và toàn bộ lời gọi API cross-origin kèm JWT bị chặn ngay ở bước OPTIONS.

CORS trong KBM

Tại thời điểm khảo sát, KBM dùng cấu hình `allowedOrigins: "*"` — cho phép *mọi* origin truy xuất API. Trong giai đoạn phát triển (development), đây là giải pháp tiện lợi để tránh lỗi CORS. Tuy nhiên, trên production, đây là một chính sách **quá rộng** so với chủ đích của KBM (API không nhằm phục vụ truy cập cross-origin công khai): mọi trang web đều có thể gọi và *đọc* phản hồi không kèm credential. Cần tách bạch hai điều thường bị nhầm: (1) với chế độ gửi kèm credential, trình duyệt *từ chối* tổ hợp `*` + `allowCredentials: true` — muốn cookie hoạt động xuyên origin bắt buộc phải allowlist origin cụ thể; (2) CORS chỉ kiểm soát việc JavaScript *đọc* response, nó **không phải** cơ chế chống CSRF — xác thực dựa trên cookie vẫn cần `SameSite`, CSRF token hoặc kiểm tra Origin riêng (Chương 9). **Lộ trình chuyển đổi (Migration path)**: (1) lập allowlist origin riêng cho từng môi trường — sách không công bố domain thật của hệ thống; (2) chỉ bật `allowCredentials: true` khi dùng xác thực cookie và đã áp dụng các biện pháp chống CSRF nói trên; (3) kiểm thử riêng với ứng dụng di động, nếu có.

8.4.4. Rate Limiting

Cơ chế giới hạn lưu lượng (rate limiting) là kỹ thuật để ngăn chặn tình trạng một client gửi quá nhiều yêu cầu trong thời gian ngắn, qua đó bảo vệ các dịch vụ cốt lõi khỏi nguy cơ quá tải hoặc bị tấn công từ chối dịch vụ (DoS). Spring Cloud Gateway hỗ trợ sẵn bộ lọc `RequestRateLimiter` kết hợp với Redis để hiện thực hóa tính năng này. Bằng cách tùy chỉnh các thông số như tốc độ bổ sung token `replenishRate` (số request mỗi giây), dung lượng tối đa của bucket `burstCapacity` (số request tối đa xử lý tức thời) và thành phần xác định khóa `KeyResolver`, hệ thống có thể kiểm soát lưu lượng truy cập dựa trên nhiều chính sách khác nhau. Các chiến lược phổ biến bao gồm:

1. **Theo người dùng (per user):** Áp dụng hạn ngạch (ví dụ: N request/giây) cho từng người dùng đã xác thực. Phương pháp này giúp ngăn chặn lạm dụng hoặc khai thác tài nguyên quá mức từ một tài khoản.
2. **Theo IP (per IP):** Giới hạn lưu lượng từ một địa chỉ IP duy nhất. Chiến lược này giúp ngăn chặn tấn công DDoS thô sơ và các bot cào dữ liệu (scraping).
3. **Theo route (per route):** Thiết lập giới hạn riêng cho từng API. Kỹ thuật này thường áp dụng để bảo vệ các API tiêu tốn nhiều năng lực tính toán hoặc I/O (ví dụ: xuất báo cáo thống kê).
4. **Toàn cục (global):** Giới hạn tổng lượng yêu cầu truy cập đi vào hệ thống từ mọi nguồn. Quy tắc này hoạt động như chiếc cầu chì an toàn cuối cùng, đảm bảo hạ tầng mạng bên trong không bị quá tải khi đối mặt với lưu lượng đột biến.

Thiết kế rate limiting với Redis:

Spring Cloud Gateway sử dụng **thuật toán token bucket** (Redis-backed) — mỗi người dùng có một "bucket" chứa thẻ bài (token), mỗi yêu cầu tiêu thụ một thẻ bài, và các thẻ bài này được bổ sung theo tốc độ `replenishRate`:

__Listing 8.4:__ Cấu hình rate limiting cho endpoint nộp bài

```yaml
# application.yml — Rate limiting cho endpoint nộp bài
spring:
  cloud:
    gateway:
      routes:
        # Đặt TRƯỚC route kbm-core ở listing route (route được xét theo thứ tự);
        # hai listing thuộc cùng một danh sách routes
        - id: kbm-core-rate-limited
          uri: lb://kbm-core
          predicates:
            - Path=/api/core/submissions/**
          filters:
            - StripPrefix=2
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 5    # 5 request/giây
                redis-rate-limiter.burstCapacity: 10   # Burst tối đa 10
                redis-rate-limiter.requestedTokens: 1  # 1 token/request
                key-resolver: "#{@userKeyResolver}"    # Rate limit theo user

  data:
    redis:
      host: localhost
      port: 6379
```

Trong cấu hình trên, bộ lọc `RequestRateLimiter` được áp dụng vào route nộp bài tập với các thông số về tốc độ bổ sung và dung lượng bucket. Tuy nhiên, Gateway cần một `KeyResolver` để xác định

định danh (khóa) cho từng yêu cầu nhằm cấp phát hạn ngạch phù hợp. Đoạn mã dưới đây triển khai `KeyResolver` dùng `X-User-Id` (do bộ lọc xác thực cung cấp), kết hợp fallback an toàn để tránh `NullPointerException` đối với các API ẩn danh:

Listing 8.5: `KeyResolver` tránh `NullPointerException` — khóa rate limit theo `userId`

```
@Configuration
public class RateLimitConfig {

    // Địa chỉ các proxy biên được phép ghi X-Forwarded-For.
    // Đưa ra cấu hình (application.yml) thay vì hardcode; danh sách rỗng
    // nghĩa là không tin header nào — mặc định an toàn.
    @Value("${kbm.gateway.trusted-proxies:}")
    private Set<String> trustedProxies = Set.of();

    @Bean
    public KeyResolver userKeyResolver() {
        return exchange -> {
            // Lấy userId từ header đã được JWT filter inject
            String userId = exchange.getRequest().getHeaders()
                .getFirst("X-User-Id");
            if (userId != null) {
                return Mono.just(userId);            // Rate limit per user
            }
            // Fallback: rate limit per IP cho anonymous requests
            // CHÚ Ý: client tự đặt được X-Forwarded-For — chỉ tin header này khi
            // proxy biên (trusted proxy) đã ghi đè/sanitize nó,
            // nếu không kẻ tấn công đổi header để né rate limit;
            // không có proxy tin cậy → dùng remoteAddress
            java.net.InetSocketAddress remoteAddress =
                    exchange.getRequest().getRemoteAddress();
            String peer =
                (remoteAddress != null && remoteAddress.getAddress() != null)
                ? remoteAddress.getAddress().getHostAddress() : null;

            // Chỉ đọc X-Forwarded-For khi request THỰC SỰ đến từ proxy biên
            // đã đăng ký.
            // Thiếu điều kiện này, client tự gửi header và tự chọn key rate limit.
            String forwardedFor = exchange.getRequest().getHeaders()
                    .getFirst("X-Forwarded-For");
            if (forwardedFor != null && peer != null
                    && trustedProxies.contains(peer)) {
                // Lấy phần tử CUỐI (IP do proxy biên ghi nhận) — phần tử trái nhất
                // là dữ liệu client tự khai, kẻ tấn công đổi tùy ý để né rate limit
                String[] hops = forwardedFor.split(",");
                return Mono.just(hops[hops.length - 1].trim());
            }
            // Không có proxy tin cậy → chỉ tin địa chỉ TCP thật, không tin header
            return Mono.just(peer != null ? peer : "anonymous");
        };
    }
}
```

Khi người dùng vượt quá hạn ngạch, Gateway tự động trả về mã lỗi **HTTP 429 Too Many Requests**, qua đó thông báo trực tiếp cho client mà không cần chuyển tiếp yêu cầu xuống các dịch vụ phía sau. Về bảo mật, `X-Forwarded-For` chỉ đáng tin khi request thực sự đi qua proxy biên đã được cấu hình ghi đè header này; khi chuỗi có nhiều hop, hãy chọn IP theo số proxy tin cậy (đếm từ phải sang) thay vì lấy phần tử trái nhất do client kiểm soát. Một lưu ý vận hành: mọi khóa dùng chung

— nhánh `"anonymous"` cuối cùng, hay thực tế hơn là cả phòng máy đi ra ngoài qua một IP NAT — đều gom nhiều người dùng vào một bucket. Nếu áp chính sách cá nhân (5 request/giây) cho bucket đó, hàng trăm sinh viên sau cùng một NAT sẽ chia nhau đúng 5 request/giây và nhận 429 hàng loạt. Route công khai cần chính sách riêng rộng hơn, hoặc đứng ngoài lớp rate limit chặt này.

Rate limit cho chế độ thi

Trong thời gian diễn ra kỳ thi, tần suất nộp bài cao hơn bình thường (hàng trăm sinh viên nộp bài cùng lúc). Cần lưu ý các điểm sau: (1) đo burst nộp bài của từng người để chọn hạn ngạch cá nhân: bucket tính theo người dùng, nên số thí sinh tăng không tự làm tăng hạn ngạch của mỗi người; (2) khi tổng số thí sinh tăng, kiểm tra năng lực xử lý của cả hệ thống, rồi thêm **giới hạn theo từng route (per-route rate limit)** dành riêng cho các API thi cử hoặc hàng đợi phù hợp; (3) tận dụng các thao tác nguyên tử của Redis để đảm bảo đếm chính xác ngay cả khi có lượng lớn request đồng thời.

8.4.5. Logging & Tracing

Gateway là chốt biên HTTP để chuẩn hóa `X-Request-ID` — mã định danh cho từng yêu cầu đi vào hệ thống, đóng vai trò mã tham chiếu hỗ trợ (support handle) khi tra cứu log hoặc đối chiếu phản hồi lỗi. Cần phân biệt rõ ba loại mã định danh (chi tiết tại Chương 11): (1) `X-Request-ID` (`requestId` trong log) định danh một HTTP request đơn lẻ ở biên; (2) `traceparent` / `traceId` (chuẩn W3C Trace Context) liên kết các span trong một vết gọi phân tán; và (3) `correlationId` đặt trong event envelope để gom nhóm các bước của cùng một luồng nghiệp vụ bất đồng bộ qua Kafka (có thể trải qua nhiều request và nhiều trace).

Tại biên mạng, Gateway *không được* chép nguyên văn giá trị `X-Request-ID` thô từ client vào log (tránh lỗ hổng *log injection* hoặc chuỗi quá dài): bộ lọc chỉ tái sử dụng giá trị hợp lệ khớp biểu thức chính quy an toàn, ngược lại sinh mã mới mang tiền tố `req-`, rồi gắn đồng thời vào cả request chuyển xuống dịch vụ phía sau lẫn response trả về client:

Listing 8.6: GlobalFilter chuẩn hóa `X-Request-ID` tại biên Gateway

```
@Component
public class RequestIdFilter implements GlobalFilter, Ordered {
    private static final String HEADER = "X-Request-ID";
    private static final java.util.regex.Pattern SAFE_ID =
        java.util.regex.Pattern.compile("^[A-Za-z0-9._-]{1,64}$");

    @Override
    public Mono<Void> filter(
            ServerWebExchange exchange, GatewayFilterChain chain) {
        String incoming = exchange.getRequest().getHeaders().getFirst(HEADER);
        String requestId = (incoming != null && SAFE_ID.matcher(incoming).matches())
            ? incoming
            : "req-" + java.util.UUID.randomUUID();

        ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
            .headers(h -> h.set(HEADER, requestId))
            .build();
        exchange.getResponse().getHeaders().set(HEADER, requestId);

        return chain.filter(
            exchange.mutate().request(mutatedRequest).build()
        );
```

```
    }

    @Override
    public int getOrder() {
        // Chạy đầu tiên, trước cả JwtAuthGlobalFilter (-1)
        return Ordered.HIGHEST_PRECEDENCE;
    }
}
```

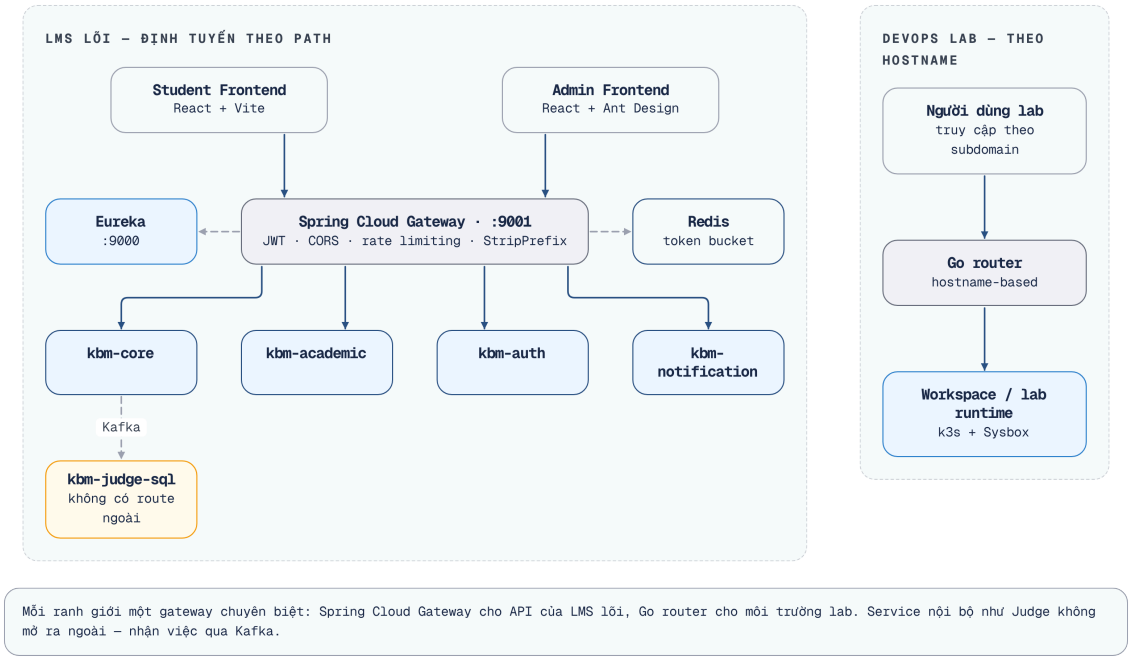
Gateway cũng là vị trí lý tưởng để ghi nhận **access log tập trung**, bao gồm: method, đường dẫn (path), mã trạng thái, độ trễ, `requestId` và định danh người dùng dẫn xuất. Thông tin này giúp hệ thống dễ dàng phát hiện endpoint nào có độ trễ bất thường, người dùng nào thường xuyên gặp lỗi và sự thay đổi của mẫu lưu lượng (traffic pattern) trước và sau khi deploy. Nhờ đó, không phải chèn thêm mã nguồn vào từng dịch vụ riêng lẻ — một `GlobalFilter` duy nhất đủ cho lớp biên HTTP.

Chi tiết triển khai structured logging, log aggregation và distributed tracing xem Chương 11 (Observability).

8.5. Case study: KBM

8.5.1. Kiến trúc tổng thể

Để hiểu rõ cách cấu hình API Gateway trong thực tế, hãy xem xét kiến trúc của hệ thống KBM (Hình 8.7). Gateway hoạt động như điểm vào duy nhất, làm nhiệm vụ định tuyến và che giấu sự phức tạp của các dịch vụ mạng nội bộ:



Hình 8.7: Kiến trúc tổng thể KBM — Gateway là điểm vào duy nhất

Hệ thống KBM có hai loại gateway/router cần được phân biệt rõ:

- **Spring Cloud Gateway**: dành cho LMS core, chịu trách nhiệm định tuyến API theo đường dẫn, xác minh JWT, cấu hình CORS, giới hạn lưu lượng và gắn mã yêu cầu ở biên (`X-Request-ID`).

- **Go hostname router**: router theo tên miền, viết bằng Go dành cho DevOps Lab, thực hiện định tuyến môi trường làm việc (workspace/lab runtime) dựa trên tên miền hoặc tên miền phụ (subdomain) nội bộ, đồng thời tích hợp trực tiếp với k3s/Sysbox ở lớp hạ tầng.

8.5.2. Phân tích cấu hình

Bảng 8.1 đối chiếu cấu hình Gateway của KBM tại thời điểm khảo sát (đầu năm 2026) với giải pháp khuyến nghị và chỉ ra khoảng trống.

Bảng 8.1: *Phân tích cấu hình Gateway trong KBM*

Khía cạnh	KBM tại thời điểm khảo sát	Giải pháp khuyến nghị	Khoảng trống
Routing	`lb://` URI qua Eureka cho các dịch vụ Java; hostname routing cho DevOps Lab	✓ Đúng theo từng ranh giới	Cần tài liệu hóa quyền sở hữu route
Xác minh danh tính	Gateway xác minh JWT; downstream có nơi tin header, có nơi tự parse token	Xác minh ở biên và ở resource service theo trust tier	Trôi lệch triển khai; cần kiểm chứng việc chặn truy cập trực tiếp
CORS	`allowedOrigins: "*"`	× Nên giới hạn	Liệt kê các origin cụ thể
Rate Limiting	Đã có: token bucket (Redis) ở Gateway	✓ Phù hợp	Chính sách riêng cho kỳ thi
SSL/TLS	Kết thúc ở lớp biên (reverse proxy hoặc Gateway)	! Tùy cách triển khai	Ghi rõ điểm kết thúc TLS trong tài liệu triển khai
Ngữ cảnh truy vết	Có nền tảng (`X-Request-ID`), chưa chuẩn hóa đầu cuối	! Nên chuẩn hóa	Chuẩn hóa `X-Request-ID` ở HTTP, `traceparent` và truyền `correlationId` qua Kafka (Chương 11)
Path Rewriting	Bộ lọc `StripPrefix`	✓ Đúng	—
WebSocket	Route tới notification service	✓ Đúng	—
Health Check	Các endpoint Actuator	✓ Đúng	—

8.5.3. Vấn đề không đồng nhất phiên bản thư viện JWT

Một vấn đề cần lưu ý trong KBM tại thời điểm khảo sát: Gateway và các dịch vụ khác dùng hai phiên bản JJWT với API khác nhau (`parserBuilder()` và `parser()`). Vấn đề không nằm ở việc resource service xác minh bằng chứng, mà ở chỗ mỗi nơi tự triển khai một chính sách khác nhau. Khi nâng cấp hoặc bổ sung RS256, sự sai lệch phiên bản (version mismatch) có thể khiến Gateway chấp nhận token nhưng service phía sau từ chối, hoặc nguy hiểm hơn là hai nơi kiểm tra issuer, audience và thuật toán không đồng nhất.

Không đồng nhất phiên bản thư viện JWT

Với thuật toán khóa đối xứng đơn giản (HS256), hai phiên bản này có thể tương thích trong luồng xử lý chuẩn (happy path). Tuy nhiên, khi nâng cấp hoặc bổ sung các tính năng phức tạp (như RS256, token refresh), sự sai lệch phiên bản sẽ gây ra các lỗi tiềm ẩn khó gỡ lỗi (debug). **Lộ trình chuyển đổi**: (1) trước mắt, đồng bộ thư viện và hợp đồng xác minh (`alg`, `iss`, `aud`, `exp`, `nbf`) để các bản triển khai không trôi lệch nhau; (2) chuyển sang Spring Security OAuth2 Resource Server thay cho `JwtUtil` tự viết; (3) dùng RS256/JWKS để verifier chỉ nhận public key và hỗ trợ luân phiên khóa; (4) khi token ngoài không đúng audience của service phía sau, Gateway thực hiện token exchange sang token nội bộ có chữ ký, không thay bằng các header văn bản thuần.

8.5.4. Lộ trình cải tiến

Giai đoạn 1 — Tăng cường bảo mật (ưu tiên cao, chi phí triển khai thấp):

- Hạn chế chặt chẽ các origin trong CORS (liệt kê các URL frontend cụ thể)

- Gỡ bỏ mọi `X-User-*` do client gửi, giữ `Authorization` cho tới khi có signed downstream token thay thế
- Thống nhất thư viện và hợp đồng xác minh `alg`/issuer/audience/expiry/not-before
- Xác minh bằng cấu hình triển khai rằng chỉ Gateway và workload được cấp phép mới gọi được service nội bộ
- Tài liệu hóa ranh giới giữa Spring Cloud Gateway và Go hostname router

Giai đoạn 2 — Observability (ưu tiên trung bình):

- Chuẩn hóa `X-Request-ID` tại Gateway (`GlobalFilter`), ngữ cảnh truy vết W3C (`traceparent`) và truyền `correlationId` qua Kafka
- Ghi log request/response tại Gateway

Giai đoạn 3 — Bảo vệ và phân tầng tin cậy (ưu tiên trung bình):

- Áp chính sách rate limiting riêng cho chế độ thi (per-route limit cho `/submissions/**`)
- Giới hạn kích thước payload cho các endpoint tải tệp lên (file upload)
- Chuyển sang RS256/JWKS và signed identity propagation đúng audience; bổ sung workload identity/mTLS cho luồng nhạy cảm

⚠ Sai lầm thường gặp

- **Đưa business logic vào Gateway**: Nếu Gateway thực hiện biến đổi dữ liệu nghiệp vụ, kiểm tra tính hợp lệ theo quy tắc nghiệp vụ (validation rule), hoặc điều phối lời gọi giữa các dịch vụ — hậu quả là Gateway sẽ biến thành một monolith mới, mọi thay đổi về business logic đều đòi hỏi phải triển khai lại toàn bộ Gateway. *Phòng tránh*: Gateway chỉ nên đảm nhiệm các nghiệp vụ cắt ngang (xác thực ở biên, CORS, routing, rate limiting); business logic thuộc về các dịch vụ phía sau.
- **Mỗi service tự xác minh JWT theo một cách khác nhau**: Nếu mỗi dịch vụ dùng một phiên bản thư viện, tập thuật toán và quy tắc issuer/audience riêng, hệ thống sẽ có lỗ hổng do các nơi kiểm tra không đồng nhất, dù cùng mang tên "JWT validation". *Phòng tránh*: Gateway xác thực ở biên; resource service xác minh bằng chứng có chữ ký bằng cấu hình hoặc middleware chuẩn hóa; mọi service thực hiện phân quyền cục bộ. Không thay bước xác minh chữ ký bằng việc tin một header văn bản thuần chỉ vì request đến từ mạng nội bộ.
- **Không có phương án dự phòng (fallback) khi Gateway ngừng hoạt động (SPOF)**: Gateway là điểm vào duy nhất nên cũng có nguy cơ trở thành điểm lỗi duy nhất (single point of failure) — khi Gateway gặp sự cố, *toàn bộ* hệ thống bị gián đoạn và không thể truy cập. *Phòng tránh*: triển khai Gateway theo mô hình tính sẵn sàng cao (High Availability — HA) bằng cách chạy nhiều instance sau một Load Balancer (ví dụ: NGINX/ALB). Đồng thời, Gateway phải tránh lỗi dây chuyền (cascading failure). Nếu Redis (dùng cho rate limiting) gặp sự cố, `RedisRateLimiter` mặc định cho request đi qua (fail-open): hệ thống vẫn chạy nhưng tạm thời không còn rate limit, nên cần giám sát và cảnh báo lỗi Redis. Ở phía client, cần triển khai cơ chế Exponential Backoff khi nhận HTTP 429.
- **CORS `allowAll` trong production**: Cho phép mọi origin đọc phản hồi API là chính sách quá rộng nếu API không có chủ đích phục vụ truy cập cross-origin công khai. Cần phân biệt hai điều: với chế độ gửi kèm credential, trình duyệt vốn đã từ chối tổ hợp wildcard `*` + credential, nên muốn xác thực bằng cookie hoạt động xuyên origin thì bắt buộc phải allowlist origin cụ thể; còn CSRF là mối đe dọa riêng, không do CORS xử lý. *Phòng tránh*: Liệt kê rõ ràng các

origin hợp lệ, và với xác thực dựa trên cookie phải bổ sung `SameSite`, CSRF token hoặc kiểm tra Origin (Chương 9).

Bổ sung bảo vệ tại Gateway: Sau khi xử lý các sai lầm trên, cần thêm các lớp bảo vệ sau:

- **JWKS (JSON Web Key Set)**: Gateway không gắn cố định public key để xác minh chữ ký JWT. Thay vào đó, Gateway tự động lấy khóa từ endpoint `/.well-known/jwks.json` của Auth Service để cho phép luân phiên khóa (key rotation) định kỳ, kết hợp cùng bộ đệm nội bộ (in-memory cache) nhằm giảm tải I/O.
- **Redis Blacklist**: Do xác minh chữ ký JWT là thao tác không trạng thái, cần đặt thời gian sống (TTL) của access token ngắn (15–60 phút, §9.2) và kết hợp Redis Blacklist cho các trường hợp thu hồi khẩn cấp (như đình chỉ tài khoản gian lận trong kỳ thi). Khi Redis gặp sự cố, cần quy định rõ đánh đổi giữa hai chế độ: *fail-open* (tạm thời chỉ dựa trên chữ ký JWT có TTL ngắn để duy trì tính sẵn sàng cho các API thông thường) và *fail-closed* (từ chối yêu cầu ở các đường dẫn nhạy cảm như nộp bài thi chính thức cho tới khi kiểm tra được trạng thái thu hồi).
- **Giới hạn kích thước (Payload Size Limit)**: Áp dụng giới hạn cứng để chặn tấn công DoS hoặc payload quá lớn trước khi chuyển tiếp request.
- **HttpOnly Cookie ở biên**: Khi Gateway hoạt động như BFF, giữ token trong *HttpOnly Cookie* thay vì đưa Bearer token cho JavaScript; Gateway chuyển cookie thành credential nội bộ khi route và vẫn áp dụng biện pháp chống CSRF phù hợp (Chương 9).

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Cơ chế định tuyến của API Gateway**: api-gateway-routing.html
- **Service Discovery & Registry**: service-discovery.html
- **Rate Limiting**: rate-limiting.html

8.6. Tổng kết

API Gateway giải quyết một thách thức cơ bản khi client tương tác với hệ thống microservices: thay vì phải biết địa chỉ của từng service, client chỉ cần biết một endpoint duy nhất. Gateway đảm nhận routing và các nghiệp vụ cắt ngang như xác thực ở biên, CORS; service phía sau tập trung vào business logic nhưng vẫn sở hữu phân quyền cục bộ và xác minh bằng chứng danh tính có chữ ký theo tầng tin cậy.

Spring Cloud Gateway (WebFlux) là lựa chọn hiện đại cho hệ sinh thái Java — reactive, non-blocking, tích hợp sâu với Spring Cloud (Eureka, Circuit Breaker). Cấu hình định tuyến với `lb://` URI kết hợp tự động với service discovery và load balancing. Nhờ đó, các service có thể mở rộng mà không cần thay đổi cấu hình tại Gateway.

Tập trung xử lý nghiệp vụ cắt ngang tại Gateway — như kiểm tra token ở biên, CORS, rate limiting và gắn `X-Request-ID` — là khoản đầu tư giúp hệ thống dễ bảo vệ, debug và vận hành hơn. Nguyên tắc cốt lõi: Gateway kiểm soát ingress; bằng chứng có chữ ký đi qua trust boundary; service đích kiểm tra bằng chứng và quyết định phân quyền cho nghiệp vụ mình sở hữu.

Phân tích KBM tại thời điểm khảo sát cho thấy cấu trúc Gateway cơ bản là phù hợp (Spring Cloud Gateway + Eureka + JWT cùng rate limiting với Redis cho LMS core, Go hostname router cho

DevOps Lab), nhưng cấu hình CORS còn quá rộng và việc truyền định danh chưa thống nhất. Lộ trình cải tiến ưu tiên làm sạch request ở biên (ingress sanitation) và một hợp đồng xác minh chung, tiếp đến RS256/JWKS cùng signed downstream identity, rồi bổ sung workload identity cho các tầng tin cậy nhạy cảm.

Ở Chương 9, chúng ta sẽ tìm hiểu sâu về **bảo mật microservices** — cấu trúc JWT, tích hợp OAuth 2.0, chiến lược luân phiên khóa hai bí mật (dual-secret key rotation), và RBAC trong KBM.

8.7. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.8: External API Patterns (API Gateway, BFF)
2. [4a] Sam Newman, *Building Microservices* — Ch.4: Integration, Smart Endpoints and Dumb Pipes
3. [1] Thomas Erl, *Service-Oriented Architecture: Analysis and Design for Services and Microservices* — API Gateway trong bối cảnh SOA doanh nghiệp

Sách bổ trợ:

4. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.8: External API Patterns (bản cập nhật)
5. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Ch.9: UI in EDA, BFF pattern

Nguồn trực tuyến:

- Spring Cloud Gateway reference — docs.spring.io/spring-cloud-gateway
- Netflix Zuul → Spring Cloud Gateway migration guide
- OWASP API Security Top 10 — owasp.org/www-project-api-security
- NIST SP 800-207 — Zero Trust Architecture
- RFC 8725 và RFC 9068 — JWT Best Current Practices và JWT Access Token Profile

Bài tập: 8-0, 8-1, 8-2 ở phần Bài tập & case study cuối sách.

Tài liệu tham khảo

[1] Erl, T. (2016). *Service-Oriented Architecture: Analysis and Design for Services and Microservices* (2nd ed.). Prentice Hall.

[2a] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

[2b] Richardson, C. (2025). *Microservices Patterns* (2nd ed.; bản MEAP, truy cập 07/2026). Manning Publications.

[4a] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.

[5] Rocha, H. F. O. (2022). *Practical Event-Driven Microservices Architecture: Building Sustainable and Highly Scalable Event-Driven Microservices*. Apress.